



ForgeMind™

A GIS Intent Compiler, Native to MCP

ForgeMind™ consumes MCP tools and exposes itself as one: a single tool a higher-level agent can hand a whole geospatial workflow to, and a structured capability surface that runs with no model call at all. Underneath, it compiles intent into a validated pipeline on ForgeGIS™ — deterministically for its templated capabilities — and refuses with a machine-readable reason when it cannot; through `run_capability`, your agent composes geospatial work without inheriting a second model's guesses.

MCP

in both directions

it drives ForgeGIS, ForgeData and the map surfaces as stdio tools, and serves itself as a tool

No-model

capability surface

list_capabilities and run_capability run a routed capability deterministically, without an LLM

Typed

refusals to branch on

a request that is not sound comes back as a named reason, not a plausible result

Delegate a workflow

Hand ForgeMind a plain-language request over MCP stdio and it resolves intent, binds data through ForgeData™, runs the pipeline and returns the result and its narration. Or skip the model entirely: `list_capabilities` enumerates what it can run, and `run_capability` runs one with structured arguments; capabilities that take a second input accept an earlier call's product.

Deterministic where it counts

The model classifies intent and extracts parameters; deterministic code owns the pipeline, so a templated capability produces the same pipeline for the same parsed intent. Requests no template covers are adapted from reference workflows by the stronger model and validated before anything runs; a low-confidence or unroutable request can fall back to a guarded agent loop whose tools you can narrow.

Any model behind it

Claude, OpenAI, Gemini and Grok are built in, each with a fast and a strong model, and `FORGEMIND_LLM_BASE_URL` points ForgeMind at any OpenAI-compatible server — Ollama is the tested path. Switching is configuration and a restart. The compiler's prompts were tuned on Claude; measure other vendors on your own workload.

BUILT FOR INTEGRATORS

Two shapes, one artifact

Run ForgeMind as an HTTP service — a buffered endpoint and a Server-Sent Events stream of progress, with health and capability endpoints and opt-in metrics — or as an MCP studio server. A bearer token, a request-size cap and a concurrency cap are built in; since 1.2.5, browser writes from any origin other than ForgeMind's own, unless you allow it, are refused.

Surfaces and scene, per request

New in 1.2.5, each request names the surface that should show the answer — ForgeGIS Studio™ or Seaglass Globe™ — and can carry the scene as structured context: entities with their symbol codes, overlays, a scenario clock. An unknown surface is refused; a deployment with no surface runs headless and answers in text.

Backends that fail cleanly

Each MCP backend sits behind a circuit breaker and respawn supervision; a slow call fails alone while a hung worker is recycled. Every inbound message is size-capped, and contour, buffer and route results too large to inline are published from the engine's file when it is in longitude and latitude — refused, never drawn as degrees, when it is not.

What the figures rest on. Zero confident-wrong answers across the June 2026 conversation campaign; about 94% of pairs passing the June 2026 compiler evaluation, on Claude; roughly a twenty-fifth of a legacy agent-loop run's model spend on one request, on Claude, where the loop hit its iteration cap. Each is scoped to its run in the Technical Brief.

Build on ForgeMind

The Technical Brief covers the architecture, both server shapes, providers, the validation record and its scope, and deployment. ForgeMind is part of the Forge suite, installed by the Seaglass Control Center™. The Forge suite is commercially licensed; evaluation builds are available to qualified teams.

rich@seaglassfoundry.com · seaglassfoundry.com